

School of Future Tech

Case Study Report

on

Programming Assignment 1 – Two-Pointer Technique

by

Satvik Pujari

Roll Number: 150096725106

School of Future Tech

Index

1. Introduction to the Case Study.

2. Problem Statement / Case Background (Abstract).

3. Problem Statement / Case Study Design.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.

5. Problem Statement / Case Study Implementation Details and Snapshots.

6. Problem Statement / Case Study Results and Conclusion.

7. References

1. Introduction to the Case Study

The Two-Pointer Technique is an algorithmic method that uses two index variables (pointers) to inspect or process elements of a data structure, most commonly an array. The pointers may move in the same direction or toward each other, depending on the problem. By making use of the order or structure of the input, this technique can often avoid repeated comparisons and reduce the amount of work performed.

This case study applies the technique to two problems involving sorted arrays: removing duplicate values in place and finding two values whose sum equals a given target. For each problem, a straightforward brute-force/reference method is compared with an optimized two-pointer method. The solutions are described in C++, Java, and Python.

The main learning outcome is to understand how pointer positions change, why each movement is safe, and how an algorithm's time and auxiliary-space complexity can be analysed.

2. Problem Statement / Case Background (Abstract)

Background

Arrays are frequently used to store collections of values. When an array is sorted, its order provides useful information: equal values appear next to one another, and values increase from left to right. Algorithms that use this property can solve some tasks more efficiently than methods that examine every possible pair or repeatedly search the array.

Problem 1 – Remove Duplicates from a Sorted Array

Given a non-decreasing integer array, modify the array in place so that each distinct value appears once in the first k positions. Return k, the number of unique values. The order of the unique values must remain unchanged. Values after index k − 1 may be ignored; the array does not need to be physically shortened.

Problem 2 – Pair Sum in a Sorted Array

Given a sorted integer array and a target value T, find two different indices i and j, where i < j, such that array[i] + array[j] = T. If no such pair exists, return (-1, -1). The two indices must refer to different elements.

Abstract

This case study implements four algorithms: brute force and two-pointer solutions for each problem. The brute-force methods provide simple reference solutions. The optimized methods use the sorted order of the input to reduce the running time. The same logic is implemented in three programming languages so that the algorithmic idea can be compared independently of language syntax.

3. Problem Statement / Case Study Design

Algorithm 1 – Brute Force: Remove Duplicates

1. Create an empty result list. 2. Visit each value in the input array from left to right. 3. Compare the current value with the values already stored in the result list. 4. If it is not present, append it. 5. Copy the result values into the beginning of the original array and return the result size. This method is easy to understand, but checking the result list for every input value can take quadratic time.

Algorithm 2 – Optimized Two Pointers: Remove Duplicates

1. If the array is empty, return 0. 2. Set a write pointer k to 1; the first element is already unique. 3. Scan the array with a read pointer i starting at 1. 4. If array[i] differs from array[k − 1], write it at array[k] and increase k. 5. Return k. The read pointer examines every element once, while the write pointer marks the end of the unique prefix.

Algorithm 3 – Brute Force: Pair Sum

1. Start with the first element. 2. Compare it with every element to its right. 3. If the two values add up to T, return their indices. 4. Continue until a pair is found or all pairs have been checked. 5. If the loops finish without a match, return (-1, -1). This checks all possible pairs and takes quadratic time in the worst case.

Algorithm 4 – Optimized Two Pointers: Pair Sum

1. Place a left pointer at the first index and a right pointer at the last index. 2. Calculate the sum of the two pointed values. 3. If the sum equals T, return the two indices. 4. If the sum is smaller than T, move the left pointer one step right to increase the sum. 5. If the sum is larger than T, move the right pointer one step left to decrease the sum. 6. Stop when the pointers meet. If no pair is found, return (-1, -1).

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Methods and logic

Brute force uses direct comparisons and is useful as a simple reference implementation. The duplicate-removal reference method checks whether each value has already been collected. The pair-sum reference method tests every pair of distinct positions.

The optimized duplicate-removal method uses a read pointer and a write pointer. Because the array is sorted, a value is new exactly when it differs from the last value written. The optimized pair-sum method uses pointers at opposite ends. Sorted order tells us which pointer to move after each comparison.

Pseudocode – Remove Duplicates (Two Pointers)

```
if n == 0: return 0\nk = 1\nfor i from 1 to n - 1:\n    if A[i] != A[k - 1]:\n
A[k] = A[i]\n        k = k + 1\nreturn k
```

Pseudocode – Pair Sum (Two Pointers)

```
left = 0; right = n - 1\nwhile left < right:\n    sum = A[left] + A[right]\n    if sum
== T: return (left, right)\n    if sum < T: left = left + 1\n    else: right = right -
1\nreturn (-1, -1)
```

Complexity analysis

Here, n represents the number of elements. Auxiliary space means extra memory used by the algorithm, excluding the input array. The brute-force duplicate-removal method uses a separate result list, while the optimized method modifies the input array in place.

Algorithm	Time	Auxiliary Space
Brute Force – Remove Duplicates	$O(n^2)$	O(n)
Two Pointers – Remove Duplicates	O(n)	O(1)
Brute Force – Pair Sum	$O(n^2)$	O(1)
Two Pointers – Pair Sum	O(n)	O(1)

Technology stack

Programming languages: C++ (C++17), Java, and Python. The implementations use standard language features and arrays/lists; no external libraries are required. The programs can be compiled or run in a standard C++ compiler, Java Development Kit (JDK), or Python 3 interpreter.

5. Problem Statement / Case Study Implementation Details and Snapshots

The assignment is organised into three source files. Each file contains implementations of all four algorithms: both brute-force/reference approaches and both optimized two-pointer approaches.

- assignment.cpp – C++ implementation of the four algorithms.

- Assignment.java – Java implementation of the four algorithms.
- assignment.py – Python implementation of the four algorithms.

Each implementation accepts or defines sample arrays and target values, runs the relevant algorithm, and displays the resulting unique prefix/count or matching pair of indices and values. The optimized duplicate-removal function returns the unique count; only the first k positions of the modified array represent the result.

Sample input and expected output

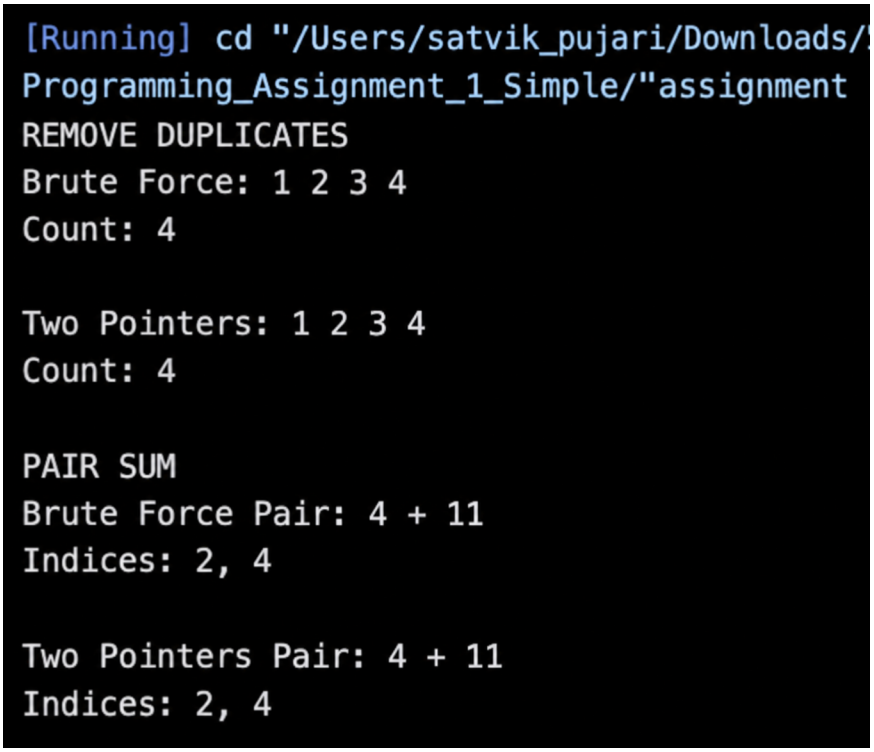
Duplicate removal: Input: [1, 1, 2, 2, 3, 4, 4]. Expected unique prefix: [1, 2, 3, 4]. Unique count: 4.

Pair sum: Input: [1, 2, 4, 7, 11, 15]. Target: 15. Expected pair: 4 + 11 = 15. Zero-based indices: 2 and 4.

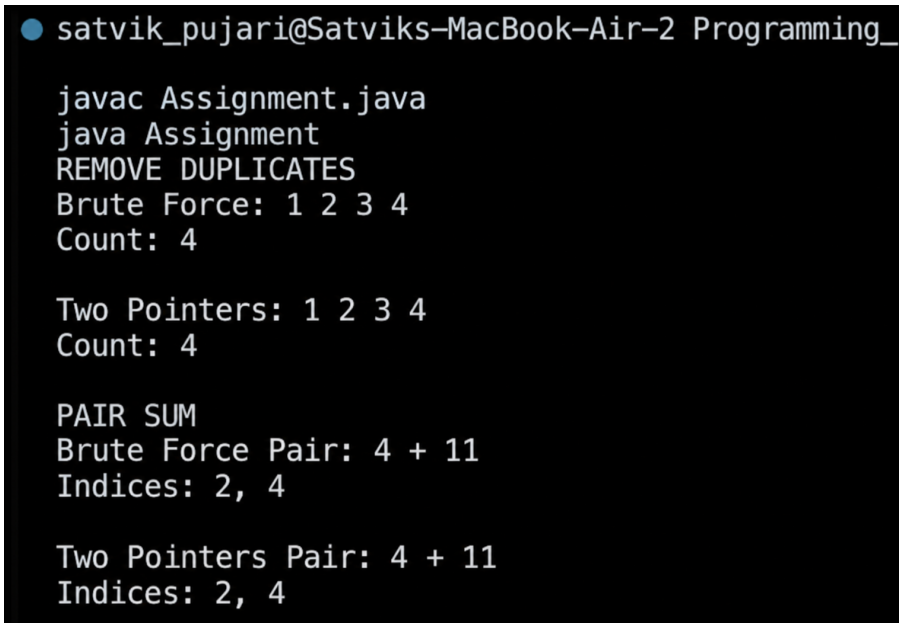
Execution snapshots

Insert genuine screenshots from the actual program execution in the labelled spaces below. The placeholders are intentional; no execution screenshots have been fabricated.

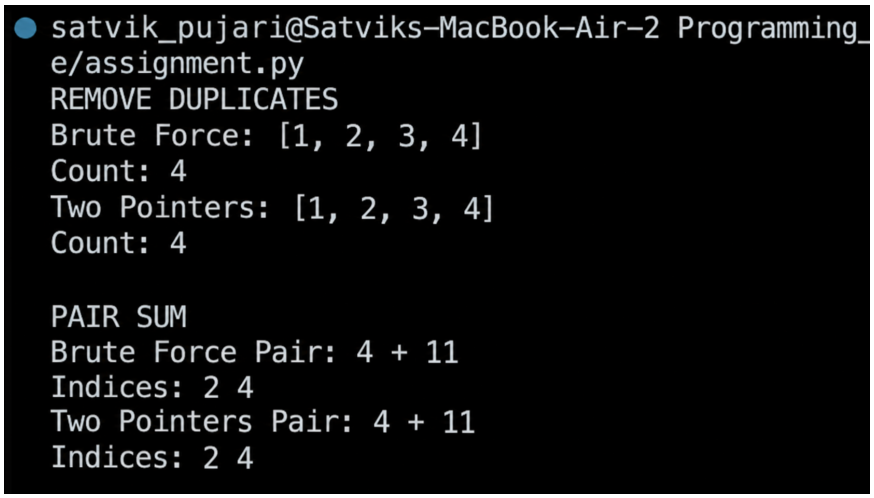
C++ Execution Screenshot



Java Execution Screenshot



Python Execution Screenshot



6. Problem Statement / Case Study Results and Conclusion

Results

For duplicate removal, the sample input [1, 1, 2, 2, 3, 4, 4] produces the unique prefix [1, 2, 3, 4], with a unique count of 4. For pair sum, the sorted input [1, 2, 4, 7, 11, 15] and target 15 produce the pair 4 and 11 at zero-based indices 2 and 4.

The brute-force and optimized approaches produce the same required results for valid inputs. Their main difference is the amount of work: brute force may perform repeated membership checks or inspect every pair, while the two-pointer algorithms make a single pass or move inward through the sorted array.

Conclusion

This case study demonstrates how the Two-Pointer Technique can improve the efficiency of array algorithms when the input is sorted. Duplicate removal improves from $O(n^2)$ time with a separate result list to O(n) time and O(1) auxiliary space by using read and write pointers. Pair sum improves from $O(n^2)$ time to O(n) time by moving the left or right pointer according to the current sum.

The technique is effective because each pointer moves in a controlled direction and does not need to revisit earlier positions. It is important to preserve the sorted-input requirement and to handle boundary cases such as an empty array, a one-element array, duplicate values, and a missing pair. Implementing the same algorithms in C++, Java, and Python reinforces that the core logic is independent of the programming language.

7. References

- C++ reference – cppreference.com, https://en.cppreference.com/w/
- Java Platform, Standard Edition Documentation – Oracle, https://docs.oracle.com/en/java/
- Python 3 Documentation – Python Software Foundation, https://docs.python.org/3/
- GeeksforGeeks – Two Pointers Technique, https://www.geeksforgeeks.org/two-pointers-technique/
- LeetCode – Remove Duplicates from Sorted Array, https://leetcode.com/problems/remove-duplicates-from-sorted-array/
- LeetCode – Two Sum II – Input Array Is Sorted, https://leetcode.com/problems/two-sum-ii-input-array-is-sorted/

Accessed for algorithm concepts and language reference material.